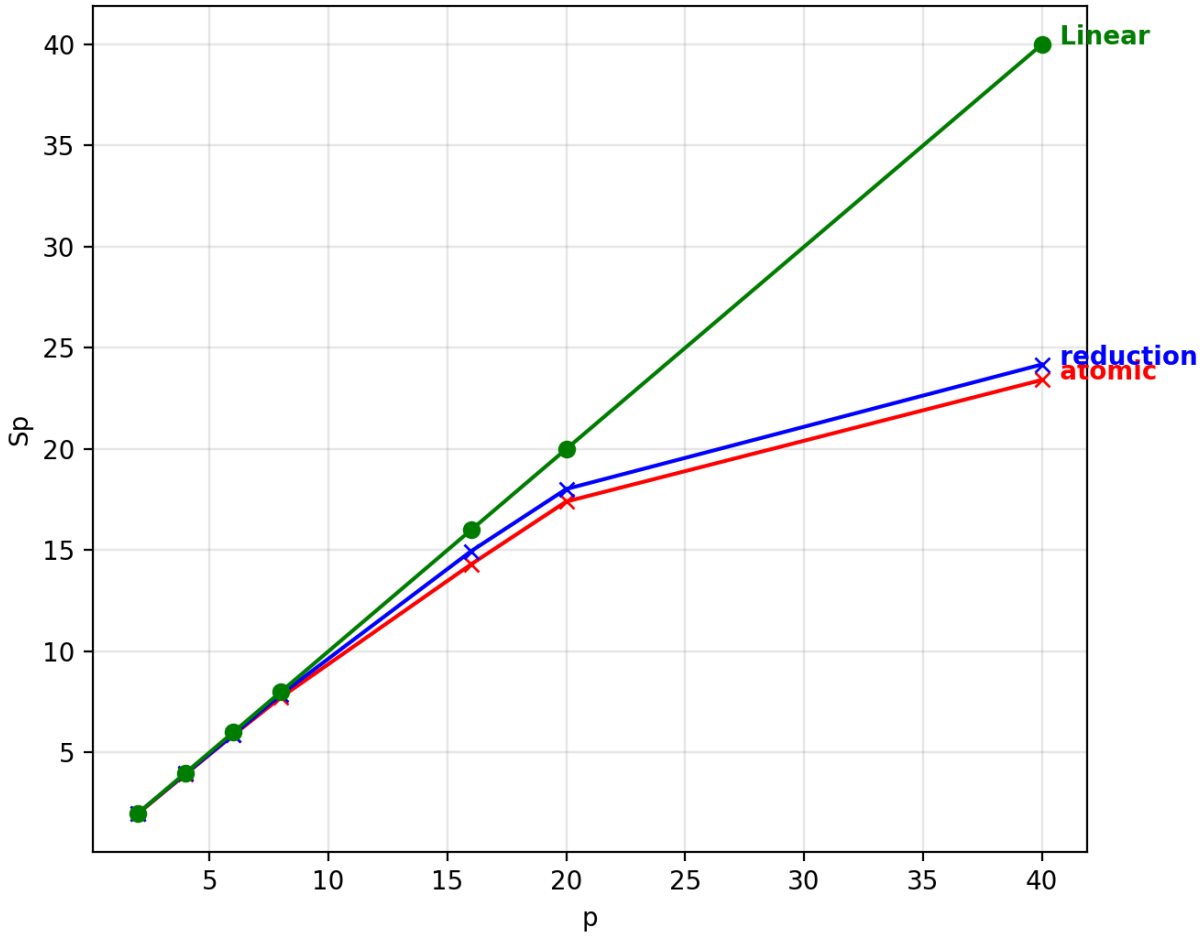
Задание 2

M × N	Количество потоков														
	2			4		6		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_6	S_6	T_8	S_8	T_{16}	S_{16}	T_{20}	S_{20}	T_{40}	S_{40}
40000 × 40000 atomic	0.48762	0.2455	1.9865	0.1241	3.9282	0.0832	5.8574	0.0631	7.7225	0.0341	14.3023	0.0280	17.4039	0.0208	23.4175
40000 × 40000 reduction	0.48762	0.2428	2.0086	0.1233	3.9558	0.0828	5.8869	0.0620	7.8658	0.0326	14.9516	0.0271	18.0231	0.0202	24.1817



В ходе работы были реализованы последовательный и параллельный алгоритмы численного интегрирования.
Наблюдается затухающий рост ускорения при увеличении числа потоков. Но если сравнивать с первым заданием, то до 20 потоков ускорение было близко к линейному. По-моему мнению это из-за меньшего количества обращений к памяти.
Задача численного интегрирования демонстрирует отличную параллельную масштабируемость благодаря отсутствию зависимостей между итерациями.
Версия с директивой reduction предпочтительнее: она проще, быстрее и надёжнее ручного разбиения. Для реальных задач рекомендуется

ограничиваться 16-20 потоками, чтобы избежать насыщения подсистемы памяти.